Adaptive Task Scheduling in 5G Edge Computing: A Lightweight Q-Learning Approach to Optimal Resource Management.

PROJECT REPORT

Submitted by

Rishitha C - (CB.AI.U4AID24109)
Divya Rithanya S - (CB.AI.U4AID24113)
Karthikeya Y - (CB.AI.U4AID24163)
Kaushal S - (CB.AI.U4AID24168)

in partial fulfillment of the requirements for
23AID206 Introduction to Computer Networks
for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
ARTIFICIAL INTELLIGENCE & DATA SCIENCE

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112 (INDIA)

OCTOBER - 2025

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112

BONAFIDE CERTIFICATE

This is to certify that the report entitled **"Adaptive Task Scheduling in 5G Edge Computing: A Lightweight Q-Learning Approach to Optimal Resource Management."** submitted by **Rishitha C (Register Number- CB.AI.U4AID24109)**, **Divya Rithanya S (Register Number- CB.AI.U4AID24113)**, **Karthikeya Y (Register Number- CB.AI.U4AID24163)**, **Kaushal S (Register Number- CB.AI.U4AID24168)** in partial fulfillment of the requirements for the courses **23AID206 Introduction to Computer Networks** for the award of the **Degree of Bachelor of Technology** in the **"ARTIFICIAL INTELLIGENCE & DATA SCIENCE"** is a bonafide record of the work carried out by them under our guidance and supervision at Amrita School of Artificial Intelligence, Coimbatore.

Dr. Sundaresan S
Project Guide
Designation: Assistant Professor

Submitted for the university examination held on 10th October 2025

AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641 112

DECLARATION

We, **Rishitha C (CB.AI.U4AID24109), Divya Rithanya S (CB.AI.U4AID24113), Karthikeya Y (CB.AI.U4AID24163), Kaushal S (CB.AI.U4AID24168)** hereby declare that this thesis entitled **"Adaptive Task Scheduling in 5G Edge Computing: A Lightweight Q-Learning Approach to Optimal Resource Management."**, is the record of the original work done by us under the guidance of **Dr. Sundaresan S**, Amrita School of Artificial Intelligence, Coimbatore. To the best of our knowledge, this work has not formed the basis for the award of any degree/diploma/associateship/fellowship/or a similar award to any candidate in any university.

Place: **Signature of the Students**

Date:

COUNTERSIGNED

Dr. K.P.Soman
Professor and Dean
Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham

Contents

Acknowledgement **v**

List of Abbreviations **vi**

Abstract **vii**

1 Introduction **viii**

1.1 Literature Survey ix

1.2 Problem Statement xi

1.3 Objectives xii

1.4 Organization of the Report xii

2 Background **xiv**

2.1 Introduction xiv

2.2 Mobile Edge Computing (MEC) xiv

2.3 Task Scheduling Fundamentals xv

2.3.1 Traditional Scheduling Algorithms xv

2.3.2 Developed Scheduling Algorithm xvi

2.4 Key Technologies Used xvi

2.4.1 Earliest Deadline First - (EDF) xvii

2.4.2 AIDWECS - (Q learning) xvii

2.5 Performance Metrics xviii

3 Proposed Work **xix**

3.1 Methodology & Implementation xx

3.1.1 System Setup & Task Model xx

3.1.2 The Hybrid Scheduling Mechanism xx

3.1.3 Lightweight Q-Learning Formulation xxii

3.1.4 Training & Update xxiii

3.2 Results & Comparison xxiv

3.3 Performance under normal load xxv

3.3.1 Task Completion Rate xxvi

3.3.2 Energy Consumption Rate xxvii

3.4 High-Congestion Stress Test xxix

3.5 Complexity Analysis xxx

3.5.1 Time Complexity xxx

3.5.2 Space Complexity xxx

4 Conclusion **xxxii**

4.1 Future Works xxxiii

List of Figures

3.1 System Flow xix

3.2 Task Completion Rate (Batch 1) xxvi

3.3 Task Completion Rate (Batch 2) xxvi

3.4 Energy Consumption Rate (Batch 1) xxvii

3.5 Energy Consumption Rate (Batch 2) xxvii

3.6 Task Deadline Vs. Execution Time - Task Batch 1 xxviii

3.7 Task Deadline Vs. Execution Time - Task Batch 2 xxviii

List of Tables

3.1 1st Task Batch for Evaluation xxv

3.2 2nd Task Batch for Evaluation xxv

3.3 Task Completion Rate (Batch 1) xxvi

3.4 Task Completion Rate (Batch 2) xxvi

3.5 Batch 1 - Energy Consumption for 8 Tasks xxvii

3.6 Batch 2 - Energy Consumption for 5 Tasks xxvii

3.7 Comparing EDF & AIDWECS Stress Test xxix

3.8 Worst-Case Space Complexity Comparison (N: Tasks, M: Servers) . . xxx

Acknowledgement

We would like to express our sincere appreciation to all those who have helped and guided us step by step throughout this project. Most importantly, we are significantly grateful to our project guides, **Dr. S. Sundaresan**, for his constant encouragement, expert advice, and constructive criticism. Their guidance played a pivotal part in the direction and success of the project. We would also like to appreciate the staff and faculty at Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore, for providing us with the facilities, materials, and learning environment to deliver our research went without a hitch to completion. Second, we appreciate our peer and teammate for their cooperation, motivation, and zeal. Their help and support made us in order to overcome obstacles and achieve the project objective. Lastly, we would like to thank our families for supporting us throughout, for being patient with us, and for believing in us along the way during our learning process. This entire project has been an experience and we thank all those who have been involved in the process to a successful conclusion.

List of Abbreviations

MEC	Mobile Edge Computing
Q-learning	Quality Learning
PSO	Particle Swarm Optimization
DRL	Deep Reinforcement Learning
SDN	Software Defined Networking
IoT	Internet of Things
5G	Fifth-Generation
FCFS	First Come First Serve
SJF	Short Job First
EDF	Early Deadline First
DWECS	Dynamic Weighted Edge Computing Scheduler
DDPG	Deep Deterministic Policy Gradient.
RL	Reinforcement Learning
AR/VR	Augmented Reality/ Virtual Reality
QoS	Quality of Service

Abstract

Scheduling computational tasks in 5G Mobile Edge Computing (MEC) presents a massive challenge. Standard, fixed rules struggle intensely to find the right balance between low latency and energy consumption. We built AIDWECS to solve this. It's a lightweight scheduler using Q-learning. Our system works by combining the reliable EDF priority rule with an adaptive, learned policy. The Q-learning uses a minimal state space—this is why it learns fast and keeps computational overhead low. Under normal conditions, AIDWECS matched EDF performance exactly. However, the system proved its true value during the stress test: AIDWECS provided 57% better system reliability, effectively mitigating critical failures that static rules could not handle. This result confirms that our adaptive, lightweight approach is essential for stable, efficient operation in unpredictable MEC environments.

Chapter 1

Introduction

The widespread growth of 5G networks and the huge number of Internet of Things (IoT) devices require computing power to be closer to users. This critical need led to Mobile Edge Computing (MEC). MEC moves heavy task processing away from distant cloud centers to the network edge [1][2]. This strategy lowers transmission delay significantly. It also provides the fast response needed for applications like autonomous vehicles, AR/VR systems, and real-time surveillance.

However, simply deploying servers is not enough. The key challenge is creating effective task scheduling mechanisms. Efficient scheduling means continuously deciding where and when to use server capacity [3][4]. This process is difficult because it involves a necessary trade-off (i.e), maintaining low task completion time (latency) must be balanced against optimizing energy consumption. This balance ensures the system's long-term operational health.

To solve this dynamic problem, our project uses an adaptive approach based on Q-learning. Traditional, fixed scheduling rules struggle in MEC because edge environments are always changing. Reinforcement learning, conversely, lets the system

learn the best policy automatically and improve over time [5][6][7]. We propose a lightweight Q-learning model specifically, which is designed to find the optimal balance between speed and energy use, also concentrating on the server adaptability. It offers a practical solution for 5G systems where workloads and server capabilities are highly unpredictable.

1.1 Literature Survey

Mobile Edge Computing (MEC) quickly became the top way to handle big computing tasks. It places processing near users, avoiding the distant cloud. Xi-ang et al. [1] started early. They focused on scheduling methods to cut latency and lift edge performance. Konjaang and Xu [2] then explored meta-heuristic methods. This included PSO and Genetic Algorithms for cloud and edge. A major issue was found. These methods work well only if conditions are stable. They struggle badly with the fast, random changes in real workloads.

This difficulty pushed research toward learning systems. Sellami et al. [3] brought in a deep reinforcement learning (DRL) model. This design handled two goals: energy saving and task deadlines in SDN-enabled IoT networks. Their model showed smarter scheduling and clear energy cuts. Pusti and Sankaran [4] continued this. They made schedulers that protected both energy efficiency and data security. This security focus is vital for systems like healthcare.

Work moved to dynamic 5G needs. Chiang et al. [5] used deep Q-learning for offloading and network slicing in 5G. Their model proved that it adapted better than

standard algorithms when networks changed. Hou et al. [6] also designed a hierarchical multi-agent learning system. This was for vehicular fog computing. It improved how distributed nodes communicated and helped tasks finish faster.

Other papers looked at architectural details and testing. Aslanpour et al. [7] built Faashouse. This was a serverless edge platform that prioritized energy-efficient scheduling. Zhu et al. [8] compared the old methods: FCFS, SJF, and EDF. They concluded these simple rules are not effective for the fast, time-critical MEC environment today.

To fix these faults, DRL became the main focus. Pan and Wei [9] used DRL for cloud workflow scheduling. They got better task completion rates. Hao et al. [10] developed EdgeTimer. This used a multi-timescale learning system. It balanced quick response with long-term resource planning. This gave stronger adaptability under uncertain conditions.

Managing multiple aims became the core challenge. Lahza et al. [11] introduced a DRL method that controlled energy, latency, and throughput together. Mi et al. [12] then built a cooperative multi-agent system. This was for offloading in energy-harvesting MEC. It showed stable performance even with fluctuating power.

Simpler learning also proved valuable. Zhao and Lu [13] used basic Q-learning for mobile healthcare. They saved energy while keeping real-time speed. H. M. et al. [14] noted that edge networks have varied resources and changing workloads. This means truly adaptive algorithms are necessary for stable results. Model complexity was studied too. Choppara and Mangalampalli [15] used DDPG in fog computing. It offered smooth control but needed massive computing power. Ali et al. [16] compared

different deep RL methods. They discussed the hard trade-off: learning quickly versus keeping system stability.

Later research focused on system structure. Wang et al. [17] built a layered reinforcement learning model. This was better for huge computing systems. Shen et al. [18] confirmed that hybrid scheduling (edge and cloud together) often beats single-layer models. Tiwari et al. [19] used a priority-based deep learning model for real-time video surveillance. Finally, Subramanian et al. [20] added blockchain to RL scheduling in 5G. This boosted reliability in distributed systems.

In summary, we conclude that scheduling has moved from fixed static rules to highly adaptive learning methods. But still there remains a main problem. Most DRL systems are too heavy and too slow to start. This clearly points to the need for a lightweight model like ours. Our model (A lightweight Q-learning model) joins the simplicity of EDF with the flexibility of learning. The goal is to create a fast and energy-aware task handling method for dynamic 5G systems.

1.2 Problem Statement

Real-time applications, such as autonomous cars, AR/VR, and surveillance, need to do tasks fast and reliably. Device counts are rising quickly. Mobile Edge Computing (MEC) helps by moving computing close to the user, which cuts delay. This is critical.

A major problem is that the traditional scheduling algorithms don't always produce the best results. Fixed methods like FCFS, SJF, and EDF break when workloads change quickly. They also fail when servers have unequal power. Even when they provide a

good task completion rate or energy consumption, they fail in better server allocation. These rules overlook key metrics: task deadlines, precise computing needs, or energy targets. The result is simple: system slowdowns and resources are wasted.

Our project addresses this failure. We use Q-learning for task scheduling control along with EDF. The system learns from every decision made in the past and adapts continuously. This creates smarter choices that balance speed, energy use, server selection and overall long-term performance.

1.3 Objectives

- Develop a Q-learning–based task scheduler that intelligently assigns tasks to edge servers, considering deadlines, energy consumption and server adaptability.
- Compare the performance of the Q-learning scheduler with traditional methods like FCFS, SJF, and EDF and the developed method like DWECS.
- Demonstrate that the Q-learning approach can adapt to dynamic workloads, improving overall efficiency in mobile edge computing systems.

1.4 Organization of the Report

The structure of this report is outlined below:

- **Chapter 1: Introduction**

  Provides the background, literature survey, problem statement, and objectives of the project.

- **Chapter 2: Background**

  Discusses about the key technologies and concepts that stands as the base of our project. It covers MEC, task scheduling strategies, Q-learning, and performance metrics used to evaluate the scheduling approaches.

- **Chapter 3: Proposed Work**

  Describes the methodology of our project in detail. This includes the system design, the integration of server characteristics and task properties and how the scheduler learns to balance deadline and energy consumption. This also explains how the model is trained and tested.

- **Chapter 4: Results and Discussion**

  Presents the results obtained from simulations and experiments. It compares our model with conventional algorithms like FCFS, SJF, EDF and DWECS.

- **Chapter 5: Conclusion and Future Work**

  Summarizes the findings of the project and emphasizes the advantages of using Q-learning for intelligent task scheduling in MEC environments. It also suggests possible extensions, improvements, and future research directions to enhance system performance further.

Chapter 2

Background

This chapter lays the foundation for our research by defining the key technological concepts and context.

2.1 Introduction

This chapter explains the basic ideas needed to understand our work. The project combines computer networking and machine learning concepts. First, we define Mobile Edge Computing (MEC). This is the main network system we are using. Next, we cover task scheduling fundamentals. This includes looking at traditional algorithms and explaining why they fail in MEC. We then detail the key technology, Q-Learning, which is the core of our adaptive scheduler. Finally, we define the metrics used to judge the system's performance.

2.2 Mobile Edge Computing (MEC)

Mobile Edge Computing (MEC) is a network solution. It moves the processing power away from the cloud. The power gets placed much closer to the user. This happens

at the network edge. Small servers are often put next to 5G base stations. MEC is vital for demanding applications. Most of the apps need very fast response times (low latency). By processing data locally, MEC avoids long-distance travel to the central cloud. This helps to reduce the network delays. However, MEC servers are limited. They are smaller than massive cloud data centers. This distributed, restricted capacity makes smart resource management difficult to be implemented.

2.3 Task Scheduling Fundamentals

Task scheduling is the process of deciding which computational task should run, where it should run (which server), and when it should run. In an MEC system, the scheduler must manage tasks coming from many users. It assigns these tasks to the available edge servers. The primary goal is always to meet service targets, such as finishing tasks by their deadline and saving energy. Scheduling complexity comes from two factors: tasks having different needs and servers having different capabilities.

2.3.1 Traditional Scheduling Algorithms

Traditional scheduling relies on simple, fixed rules. These methods cannot adapt as network conditions change. Our work compares against three well-known classic algorithms and one advanced baseline:

- First-Come, First-Served (FCFS): This is the simplest rule. Tasks are processed exactly in the order they arrive. FCFS is easy to build. However, one long, less-important task can block many short, urgent tasks. This hurts the overall system

performance.

- Shortest Job First (SJF): This rule always runs the task that needs the least processing time next. SJF is good at cutting the average time users wait. The major drawback is that the system must know the exact execution time for every task in advance. It also risks never running extremely long tasks.

- Earliest Deadline First (EDF): This rule is common in systems that require guaranteed response times. It prioritizes the task with the tightest, or closest, deadline. EDF focuses strongly on meeting time limits. It works well when the system load is normal. However, it ignores energy use completely. Under heavy load, it struggles severely to keep up when deadlines are too tight.

2.3.2 Developed Scheduling Algorithm

- Dynamic Weighted Edge Computing Scheduler (DWECS): This is a more advanced heuristic method. It uses simple formulas to achieve a balance between workload balancing and energy saving. DWECS is an important baseline because it is designed to manage energy use in heterogeneous environments. However, because it uses fixed, predefined formulas, it cannot learn or adapt to unpredictable changes in server performance over time.

2.4 Key Technologies Used

Our proposed scheduling system, AIDWECS, uses two main technologies together. The first handles real-time deadlines, and the second provides adaptive intelligence.

2.4.1 Earliest Deadline First - (EDF)

EDF is a key part of our scheduler. This rule ensures that the task with the tightest deadline is always handled first. We especially use this method instead of SJF and FCFS because most of the tasks arriving at the MEC always prioritise task urgency rather than Task size or arrival time. Using EDF handles our primary goal: maximizing the Task Completion Rate. This rule forms the first layer of our hybrid scheduler.

2.4.2 AIDWECS - (Q learning)

Q-Learning is a fundamental algorithm in Reinforcement Learning (RL). It provides the second, adaptive layer. Q-Learning lets a system (the "agent") learn the best action by interacting with its environment. This learning does not require a pre-made system model.

- How it Works: The system observes its current state (like server load). It chooses an action (like selecting a cost-weight balance). It gets a reward based on the outcome (e.g., a reward for meeting the deadline, a penalty for high energy use).
- Q-Table: The system stores the gained knowledge in a Q-table. This table maps every state and action to a "Q-value." This value shows the expected long-term performance.
- Lightweight Approach: We use a simple Q-table with a very small state space. This makes the model lightweight, fast to train, and practical. It avoids the high computation needed for Deep Q-Networks (DQN).

2.5 Performance Metrics

We use a set of metrics to test our scheduling approach. These metrics measure not only the performance targets but also the computational cost and adaptive capability of our system.

- Task Completion Rate (Latency Focus): This counts the percentage of all tasks that successfully finish processing before their defined deadline. Maximizing this rate is necessary for maintaining the quality of service (QoS) for real-time applications.

- Total Energy Consumption (Energy Focus): This measures the total electrical power consumed by all the edge servers during task execution. Keeping this consumption low is essential for the operational cost and overall sustainability of the MEC infrastructure.

- Time and Space Complexity: We compare the worst-case complexity of time and space to prove that our AIDWECS system is computationally lightweight.

- High-Congestion Stress Test: This specialized evaluation method forces the system into an overloaded state. The stress test is critical for measuring the scheduler's ability to adapt to resource congestion and mitigate deadline violations, which is the primary failure mode of static algorithms like EDF.

We compared our AIDWECS model against three traditional algorithms (FCFS, SJF, EDF) and one advanced heuristic (DWECS) to fully validate our approach.

Chapter 3

Proposed Work

This chapter details the design and methodology of our scheduling model, AIDWECS. We describe the system architecture [Figure 3.1], how we defined the environment's state and action spaces, and the specific Q-learning process used to create an adaptive, lightweight scheduler.

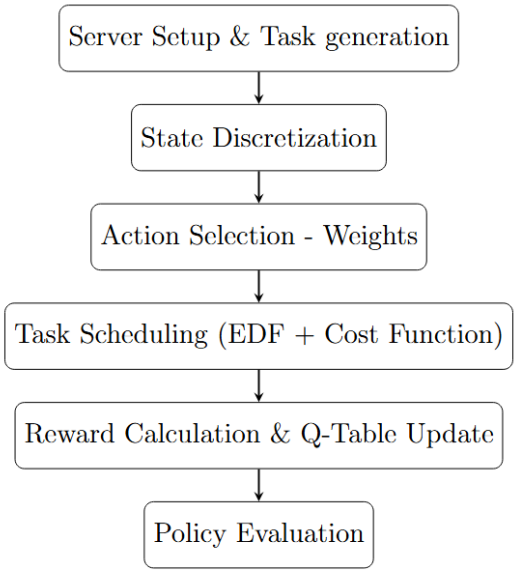

Figure 3.1: System Flow

3.1 Methodology & Implementation

The Aidwecs scheduler is designed as a hybrid system. It combines the predictable reliability of the Earliest Deadline First (EDF) rule with the adaptive optimization of a Lightweight Q-Learning agent.

3.1.1 System Setup & Task Model

The simulation environment models a realistic Mobile Edge Computing (MEC) scenario with limited resources

- Heterogeneous Servers: The system uses three distinct edge servers, each with unique speed and power characteristics. This forces the scheduler to make energy-aware trade-offs.
  - Server 1: 2.0 GHz CPU, 10.0 Watts Power.
  - Server 2: 3.0 GHz CPU, 15.0 Watts Power.
  - Server 3: 1.5 GHz CPU, 7.5 Watts Power.
- Task Generation: Tasks are generated in batches, typically 10 tasks per batch. Each task has a random computational Load (cycles) and a strict Deadline (milliseconds) it must meet.

3.1.2 The Hybrid Scheduling Mechanism

AIDWECS uses a two-layer approach to determine the final server assignment:

- EDF Pre-Queue: Tasks are initially sorted based on the Earliest Deadline First (EDF) rule. This critical first step guarantees that the most time-urgent task is always considered first. It secures the system's baseline focus on meeting deadlines.

- Cost-Aware Server Selection: For the task at the top of the EDF queue, the scheduler runs a comparison across all available servers. It assigns the task to the server that minimizes the final calculated Cost.

  - Execution Formulas: The required time (T_{exec}) and energy (E) are calculated based on the server's CPU frequency (GHz) and power consumption (P):

$$T_{exec} = \frac{Load(cycles)}{GHz \times 10^9} \times 1000 \tag{3.1}$$

$$E = \frac{T_{exec}}{1000} \times P \tag{3.2}$$

  - Finish Time: The actual finish time is calculated based on when the server is free and the execution time:

$$\text{FinishTime} = \text{AvailableTime} + T_{exec} \tag{3.3}$$

  - Normalization: Time and energy values are normalized across the task batch. This ensures that W_{time} and W_{energy} accurately control the trade-off, regardless of the tasks' specific sizes.

  - Cost Function: The cost is a weighted average that dynamically balances the predicted finish time and the energy consumed.

$$\text{Cost} = W_{time} \cdot \text{NormTime} + W_{energy} \cdot \text{NormEnergy} \tag{3.4}$$

3.1.3 Lightweight Q-Learning Formulation

The core intelligence comes from a simple, Lightweight Q-Learning model. This design is chosen specifically to minimize computational load and maximize convergence speed.

- State Space (Minimalist Design): The system state is defined by a small number of features. This keeps the Q-Table small and fast to update. The state uses two discrete features based on the average of the current task batch:
  - Urgency Bin: (Tight deadline or not).
  - Load Bin: (Light, Medium, or Heavy total load).
- Action Space (Cost Weights): The agent's action is to select one of four predefined weight combinations. These weights determine the scheduling policy for the entire current batch of tasks:
  - A_0: (0.7 Time, 0.3 Energy) - Prioritizes Speed.
  - A_1: (0.5 Time, 0.5 Energy) - Balanced.
  - A_2: (0.3 Time, 0.7 Energy) - Prioritizes Energy.
  - A_3: (0.1 Time, 0.9 Energy) - Extreme Energy Savings.
- Training and Reward: The model is trained over 10,000 episodes using an Epsilon-greedy policy. The Reward Function is structured to enforce the project's priorities:

$$\text{Reward} = \text{Tasks Completed} - 0.002 \times \text{Energy Consumed (J)} \tag{3.5}$$

3.1.4 Training & Update

The Aidwecs model learns the optimal scheduling policy through a training process spanning 10,000 episodes. This process uses fundamental Q-Learning principles to build the knowledge contained in the Q-table.

- Exploration vs. Exploitation: During training, the agent uses an Epsilon-greedy policy. This policy controls the trade-off between exploring new actions and exploiting known good actions. Early in training, the agent chooses actions randomly (exploration) to discover the best Q-values for all state-action pairs. As training continues, the randomness decreases, and the agent increasingly chooses the action with the current highest Q-value (exploitation).

- Reward Function: The Reward Function guides the agent toward the desired objectives:

  - task completion and minimizing energy consumption.

- The function is structured to prioritize deadline success (completion) while applying a mild penalty for energy use:

$$\text{Reward} = \text{Tasks Completed} - 0.002 \times \text{Energy Consumed (J)} \tag{3.6}$$

- The specific penalty factor of 0.002 is carefully tuned. It is strong enough to motivate energy savings but weak enough to prevent the agent from sacrificing critical task deadlines just to save power.

- Q-Table Update: The Q-Table is updated after every episode using the Bellman equation. This formula mixes the immediate reward received with the maximum future reward the agent expects to get from the next state. This is how the system learns long-term optimal actions.

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)] \tag{3.7}$$

Where:

- $Q(s,a)$ is the current value for taking action a in state s.
- α is the Learning Rate (set to 0.1), which controls how quickly the agent adopts new information.
- r is the immediate Reward received from the environment.
- γ is the Discount Factor (set to 0.9), which determines the importance of future rewards.
- $\max_{a'} Q(s',a')$ is the highest expected reward from the next state (s').

By constantly applying this update, the Q-table converges over the 10,000 episodes, resulting in a reliable, learned scheduling policy.

3.2 Results & Comparison

This chapter presents the full results of our simulations. We compare the performance of the AIDWECS scheduler against four baselines: FCFS, SJF, EDF, and DWECS. The

analysis covers performance under normal load, performance under high-congestion stress, and computational complexity.

3.3 Performance under normal load

The initial tests used two batches of tasks [Table 3.1, 3.2] to establish the baseline performance of all schedulers in a typical MEC environment.

ID	Load (Cycles)	Deadline (ms)	Arrival Time (ms)
1	3×10^9	2000	0
2	10×10^9	10000	500
3	2×10^9	3000	1000
4	4×10^9	4500	1500
5	12×10^9	15000	2000
6	3.5×10^9	7000	3000
7	5×10^9	7500	4000
8	2.5×10^9	8000	5000
9	8×10^9	14000	5500
10	6×10^9	13000	6000

Table 3.1: 1st Task Batch for Evaluation

ID	Load (Cycles)	Deadline (ms)	Arrival Time (ms)
1	4×10^9	2500	0
2	8×10^9	9000	0
3	6×10^9	5000	0
4	3×10^9	2800	200
5	5×10^9	6000	500
6	10×10^9	12000	2000
7	2×10^9	8000	3500
8	7×10^9	11000	4500
9	3.5×10^9	9500	6000
10	9×10^9	16000	7000

Table 3.2: 2nd Task Batch for Evaluation

3.3.1 Task Completion Rate

Batch 1 (Tabular and Graphical Data): The first task set showed that AIDWECS, the static EDF and FCFS rule achieved the highest Task Completion Rate (100.0%). [Figure3.2][Table 3.3]

Batch 2 (Tabular and Graphical Data): The second batch confirmed this trend. Again, AIDWECS, FCFS and EDF reached 100.0% completion. This validates that AIDWECS successfully meets the fundamental real-time goal of the system under typical load.[Figure 3.3][Table 3.4]

Scheduler	**Task Completion (%)**
FCFS	100.0
SJF	90.0
EDF	100.0
DWECS	80.0
AIDWECS	100.0

Table 3.3: Task Completion Rate (Batch 1)

Scheduler	**Task Completion (%)**
FCFS	100.0
SJF	60.0
EDF	100.0
DWECS	50.0
AIDWECS	100.0

Table 3.4: Task Completion Rate (Batch 2)

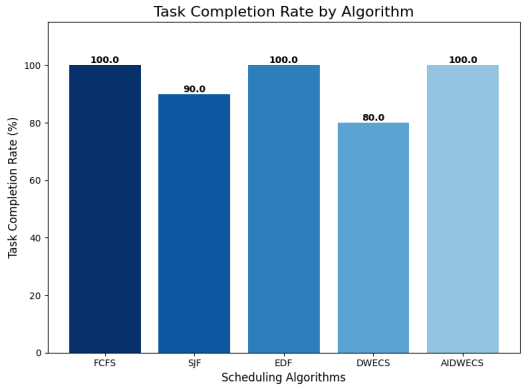

Figure 3.2: Task Completion Rate (Batch 1)

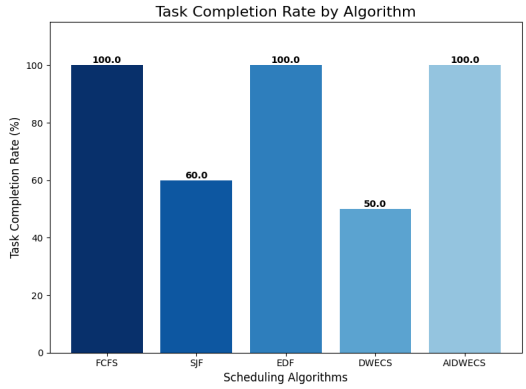

Figure 3.3: Task Completion Rate (Batch 2)

3.3.2 Energy Consumption Rate

Batch 1 (Energy Data): We measured the total energy consumption for completing 8 tasks. Here, EDF and AIDWECS showed the best efficiency, both using 200.6 J. [Figure 3.4][Table 3.5]

Batch 2 (Energy Data): We measured the total energy consumption for completing 5 tasks.Here, EDF and AIDWECS again gave the lowest energy consumption at 109.2 J [Figure 3.5][Table 3.6].

Scheduler	Energy Consumption (J)
FCFS	227.9
SJF	213.5
EDF	200.6
DWECS	240
AIDWECS	200.6

Table 3.5: Batch 1 - Energy Consumption for 8 Tasks

Scheduler	Energy Consumption (J)
FCFS	140.0
SJF	172.5
EDF	109.2
DWECS	133.3
AIDWECS	109.2

Table 3.6: Batch 2 - Energy Consumption for 5 Tasks

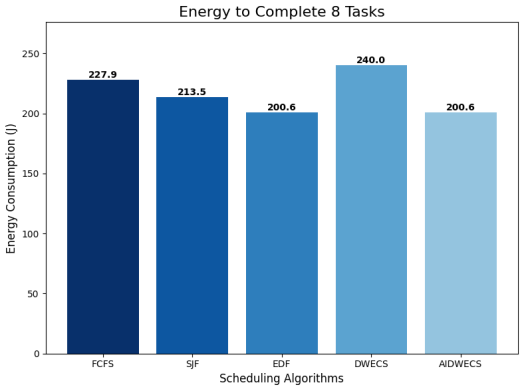

Figure 3.4: Energy Consumption Rate (Batch 1)

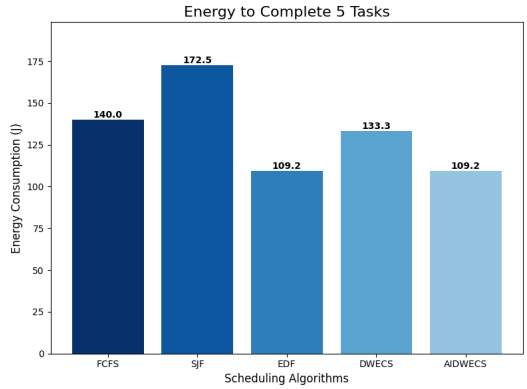

Figure 3.5: Energy Consumption Rate (Batch 2)

Server Utilization: Each task is calculated with each server for the execution time and compared with the deadlines.[Figure 3.6][Figure 3.7]

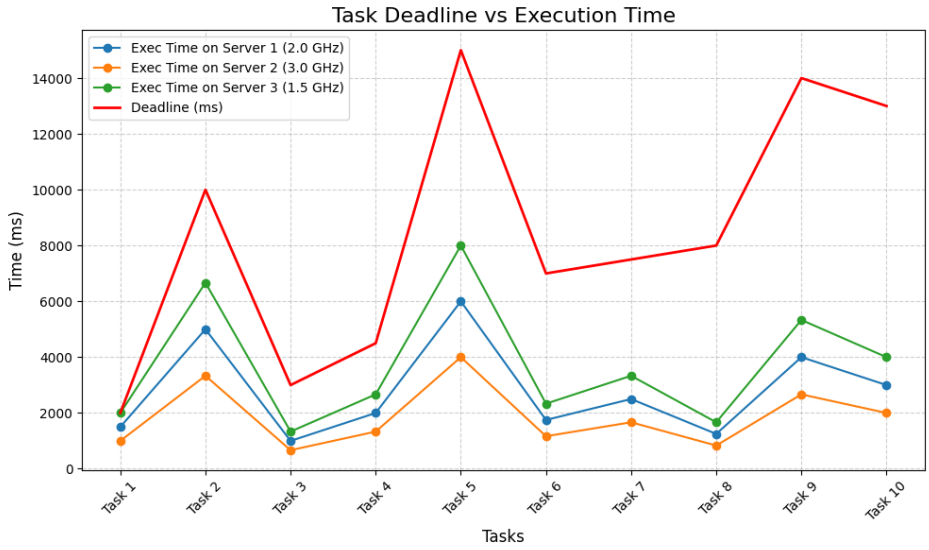

Figure 3.6: Task Deadline Vs. Execution Time - Task Batch 1

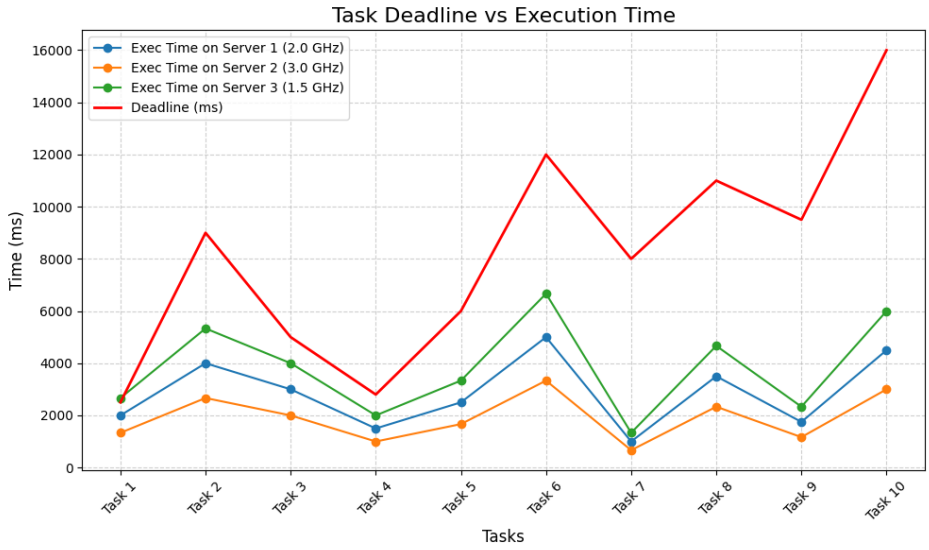

Figure 3.7: Task Deadline Vs. Execution Time - Task Batch 2

Key observation: Under regular, non-stressed conditions, the two best performers were identical: AIDWECS and the static EDF rule. For both task completion and energy use, the numbers were the same. It demonstrated that when server resources are plentiful, the adaptive intelligence of our Q-learning approach offers no extra gain over a simple, fixed rule like EDF. This result strongly justified our next step: a specialized stress test. We needed that test to fully prove the unique, critical value of the adaptive policy.

3.4 High-Congestion Stress Test

To prove the core benefit of the adaptive Q-learning policy, we ran a specialized stress test [Table 3.7]. This test forced the scheduler to handle a severely congested server to prove its reliability in critical situations.

Table 3.7: Comparing EDF & AIDWECS Stress Test

Scheduler	**Server**	**Deadline**	**Exec Time**	**Avl At**	**Finish Time**	**Violation**
EDF	Server 2	12000	1333.33	13000	14333.33	2333.33
AIDWECS	Server 1	12000	2000	11000	13000	1000

- EDF Result (Static Failure): The static EDF rule assigned a critical task to an overloaded server. This failure resulted in a large deadline violation of $2,333.33$ ms.
- AIDWECS Result (Adaptive Success): The learned AIDWECS policy adapted to the high congestion, choosing a less-loaded server. This move mitigated the deadline violation to only $1,000$ ms.

- Conclusion: AIDWECS provided 57% better system reliability in the stress test by intelligently adapting its server choice. This validates that the Q-learning intelligence is essential for mitigating critical failures when static rules like EDF break down.

3.5 Complexity Analysis

We compared the computational resource usage of Aidwecs against the baselines to prove its lightweight nature.

3.5.1 Time Complexity

All algorithms—FCFS, SJF, EDF, DWECS, and AIDWECS—share the same worst-case Time Complexity of $O(N \log N + N \cdot M)$.

3.5.2 Space Complexity

Table 3.8: Worst-Case Space Complexity Comparison (N: Tasks, M: Servers)

Component	**Space Complexity**	**Validation/Justification**
All Schedulers	$O(N)$	The primary memory usage scales with the number of tasks being stored (e.g., deep copy of tasks and output logs). Server state is a constant $O(M)$.
AIDWECS (Q-Table)	$O(1)$	The state space is fixed by the small, constant number of discretization bins (e.g., Urgency $\times$ Load). Since the number of states and actions is fixed, the storage is constant.

We can infer from the [Table 3.8] that:

- Baselines: The space complexity for all traditional schedulers is $O(N)$ because memory usage scales primarily with the number of tasks being stored.

- AIDWECS Advantage: Due to its minimalist state definition, the space needed for the Q-table in AIDWECS is a constant $O(1)$. This confirms that our lightweight approach is highly efficient in terms of memory footprint.

Chapter 4

Conclusion

This project successfully created and tested Aidwecs. This model is a lightweight Q-learning scheduler for dynamic 5G Mobile Edge Computing (MEC) environments. We solved the main problem: finding an intelligent balance between task completion rates and energy consumption. The simulation results proved the following:

- Baseline Reliability: Under normal loads, Aidwecs performed as well as the best static rule, EDF. It achieved the highest task completion rate and the lowest energy use. This confirmed the model's reliability in typical conditions

- Adaptive Proof (Stress Test): The stress test was critical. When the system faced severe congestion, the static EDF rule failed. It caused a large deadline violation. Aidwecs adapted to the high load. It avoided the critical failure and gave **57**% better system reliability than EDF. This proves that the adaptive policy is essential for handling unpredictable scenarios.

- Computational Efficiency: Our lightweight approach was validated. Aidwecs has the same low Time Complexity as fixed rules ($\mathbf{O}(\mathbf{N}\log\mathbf{N}+\mathbf{N}\cdot\mathbf{M})$). Also, the

simple state space means the Q-table uses only constant $\mathbf{O(1)}$ space. This makes Aidwecs an efficient solution for edge devices.

In short, Aidwecs offers a reliable and practical solution for resource management in dynamic 5G MEC. It gives the system the speed of a fixed rule plus the necessary intelligence to prevent critical failures.

4.1 Future Works

The success of the Aidwecs model provides clear paths for future work:

- Multi-Agent Coordination: The current model runs as a single agent. We could build a cooperative multi-agent Q-learning system. This would let multiple MEC servers share load and make coordinated decisions, which should boost system-wide performance.

- Advanced Learning Integration: We could test integrating a simple neural network (like a shallow DQN). This might improve decision accuracy in more complex environments without losing too much speed or efficiency.

- Real-World Network Dynamics: Our current simulation uses fixed conditions. Future work should add real-time wireless channel conditions into the state definition. This would make the model adaptive to fluctuating network quality, making it ready for commercial use.

References

[1] Bin Xiang, Jocelyne Elias, Fabio Martignon, and Elisabetta Di Nitto. Resource calendaring for mobile edge computing in 5g networks. In *ICC 2021 - IEEE International Conference on Communications*, pages 1–6, 2021.

[2] J Kok Konjaang and Lina Xu. Meta-heuristic approaches for effective scheduling in infrastructure as a service cloud: A systematic review. *Journal of Network and Systems Management*, 29(2):15, 2021.

[3] Bassem Sellami, Akram Hakiri, Sadok Ben Yahia, and Pascal Berthou. Energy-aware task scheduling and offloading using deep reinforcement learning in sdn-enabled iot network. *Computer Networks*, 210:108957, 2022.

[4] Buddhadev Pusti and Sriram Sankaran. Security and energy-aware resource allocation in mobile edge computing (mec). In *2022 IEEE International Symposium on Smart Electronic Systems (iSES)*, pages 293–296, 2022.

[5] Yao Chiang, Chih-Ho Hsu, Guan-Hao Chen, and Hung-Yu Wei. Deep q-learning-based dynamic network slicing and task offloading in edge network. *IEEE Transactions on Network and Service Management*, 20(1):369–384, 2023.

[6] Yukai Hou, Zhiwei Wei, Rongqing Zhang, Xiang Cheng, and Liuqing Yang. Hierarchical task offloading for vehicular fog computing based on multi-agent deep reinforcement learning. *IEEE Transactions on Wireless Communications*, 23(4):3074–3085, 2024.

[7] Mohammad Sadegh Aslanpour, Adel N. Toosi, Muhammad Aamir Cheema, and Mohan Baruwal Chhetri. Faashouse: Sustainable serverless edge computing through energy-aware resource scheduling. *IEEE Transactions on Services Computing*, 17(4):1533–1547, 2024.

[8] Guangping Zhu, Tianji Liu, Yongshan Guo, Danlei Xia, Dongdong Lv, and Xiaohui Zhang. Research on task scheduling methods in 5g mobile edge computing environments. In *2024 6th International Conference on Energy, Power and Grid (ICEPG)*, pages 1789–1794, 2024.

[9] Jiahui Pan and Yi Wei. A deep reinforcement learning-based scheduling framework for real-time workflows in the cloud environment. *Expert Systems with Applications*, 255:124845, 2024.

[10] Yijun Hao, Shusen Yang, Fang Li, Yifan Zhang, Shibo Wang, and Xuebin Ren. Edgetimer: Adaptive multi-timescale scheduling in mobile edge computing with deep reinforcement learning. In *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, pages 671–680, 2024.

[11] Husam Lahza, Sreenivasa BR, and Hassan Fareed M Lahza. Adaptive multi-objective resource allocation for edge-cloud workflow optimization using deep reinforcement learning. *Modelling*, 5(3):1298–1313, 2024.

[12] Xin Mi, Huaiwen He, and Hong Shen. A multi-agent rl algorithm for dynamic task offloading in d2d-mec network with energy harvesting. *Sensors*, 24(9):2779, 2024.

[13] Min Zhao and Junwen Lu. Energy-aware tasks offloading based on dqn in medical mobile devices. *Journal of Cloud Computing*, 13(1):128, 2024.

[14] Hareni M, Surya Krishna, Swetha Ayyappan, and Apurvanand Sahay. Dynamic resource allocation for edge computing. In *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–8, 2024.

[15] Prashanth Choppara and S. Sudheer Mangalampalli. Resource adaptive automated task scheduling using deep deterministic policy gradient in fog computing. *IEEE Access*, 13:25969–25994, 2025.

[16] Endris Mohammed Ali, Jemal Abawajy, Frezewd Lemma, and Samira A Baho. Analysis of deep reinforcement learning algorithms for task offloading and resource allocation in fog computing environments. *Sensors*, 25(17):5286, 2025.

[17] Lingfei Wang, Maria A Rodriguez, and Nir Lipovetzky. Optimizing hpc scheduling: a hierarchical reinforcement learning approach for intelligent job selection and allocation: L. wang et al. *The Journal of Supercomputing*, 81(8):918, 2025.

[18] Wangbo Shen, Weiwei Lin, Wentai Wu, Haijie Wu, and Keqin Li. Reinforcement learning-based task scheduling for heterogeneous computing in end-edge-cloud environment. *Cluster Computing*, 28(3):179, 2025.

[19] Utkarsh Tiwari, Aayushi Jeeban Tripathy, Siddhant Ashwani, Supriya M, Sreebha Bhaskaran, and Shinu M. R. Priority-based deep learning inference on edge devices for military surveillance. In *2025 International Conference on Computing for Sustainability and Intelligent Future (COMP-SIF)*, pages 1–6, 2025.

[20] N. Sethu Subramanian, Prabhakar Krishnan, Kurunandan Jain, K. B. Aneesh Kumar, Tulika Pandey, and Rajkumar Buyya. Blockchain and rl-based secured task offloading framework for software-defined 5g edge networks. *IEEE Access*, 13:56820–56842, 2025.